

Scaling reasoning; De Bruijn graphs enable generalization from sparse and length-limited data

We want to learn many reasoning paths and longer reasoning paths from few and length-limited samples.

Number of samples you need to learn a set of reasoning paths scales with number of edges in the minimal DAG

De Bruijn structure is excellent at compressing a large number of paths into a small DAG

This allows for high sample efficiency and length generalization (e.g., in synthetic data sampled from random graphs as well as in a navigation task)

Actual reasoning traces also have strong dependence on just their local context

Samples with this structure tend to have higher uncertainty at the beginning of reasoning - mimicking the initial uncertain "exploration" phase at the beginning of reasoning.

Introduction

Equipping large language models (LLMs) with the ability to execute long-horizon tasks through extended planning and reasoning remains a primary frontier in artificial intelligence. Success in these domains relies on a model's capacity to produce lengthy and diverse reasoning traces. In Appendix A, we build on the work of [Nadgir et al.] to show that these kinds of extended reasoning traces decompose non-smooth tasks, where minor input changes change the output, into a sequence of smooth predictions that are easier to learn.

In practice, models already exhibit this behavior; for instance, DeepSeek-R1's reasoning traces for basic grade-school math average $[X]$ tokens. This length introduces a severe sample complexity problem. The space of possible reasoning paths grows exponentially with reasoning length, dwarfing the scale of any feasible pre-training corpus.

It is therefore impossible for training data to meaningfully sample the distribution of valid reasoning sequences. Yet, we would like our LLMs to generalize to novel, long-horizon tasks. In order for this to be possible, different reasoning trajectories should share subsequences in a structured way that increases the sample-efficiency of learning. We propose that this generalization requires local structure within the reasoning traces: the distribution of valid next steps is constrained primarily by recent context, decoupling an individual reasoning step from the full past trajectory. The true transition dynamics of the logical task should be approximated by considering the recent n -gram state, which a LLM can exploit.

While there are many similar structures that can also compress the set of reasoning paths, e.g., if next-tokens are generated by an underlying finite automaton, our experimental results show that LLMs can struggle to learn this underlying structure unless it manifests as n -gram transitions in the data. Learning longer-range correlations in the tokens requires more data.

Similar results have been observed in prior work which claims that next-token prediction is not an effective target for recovering the underlying world-model (finite automaton) of a task [https://proceedings.neurips.cc/paper_files/paper/2024/file/2f6a6317bada76b26a4f61bb70a7db59-Paper-Conference.pdf]. In addition, [https://arxiv.org/abs/2602.07488] derive precise scaling exponents for the test loss of a LLM during training using the hypothesis that LLMs trained on finite data can only learn up to n -gram correlations in the text.

All the code to recreate these experiments is on GitHub [https://github.com/amrutn/debruijn_experiments].

Section 1: Local Structure in Reasoning Traces Enables Generalization with Limited Samples

The complete set of valid reasoning traces for any task can be organized in a prefix tree. At any node in the tree, outgoing branches represent the set of plausible next reasoning steps. While decoding, the LLM uses the prompt to select the correct next step from this set. Each complete path from root to leaf represents a valid reasoning trace. [Nadgir et al.] show that by navigating this deep prefix tree, a LLM transforms a single non-smooth task into a sequence of smooth, easy-to-predict intermediate problems. Executing this decomposition requires learning every transition in the prefix tree. We will investigate how a LLM learns all these transitions despite observing only a sparse or length-limited sample of complete paths during training.

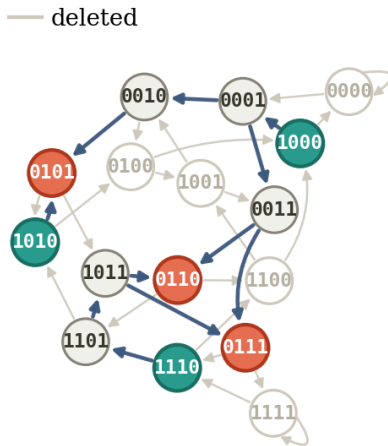


Fig 1: Visualization of a De Bruijn graph Edges show transitions from one 4-gram to another one by adding a single token. Deleted edges and nodes were pruned when applying the procedure F to the graph to create a directed acyclic graph (DAG) from its edges. Green shaded nodes are start (source) nodes and orange shaded nodes are answer (sink) nodes in the resulting DAG.

This kind of generalization requires that the prefix tree of reasoning traces is compressible into a much smaller set of transition rules. Shannon originally demonstrated that text is compressible because the conditional entropy of natural language decays as the conditioning context grows [Shannon, 1951, Prediction and Entropy of Printed English]. Furthermore,

because pairwise token correlations in text decay with separation [https://arxiv.org/abs/2602.07488], the next-token prediction of an LLM is largely constrained by its local n -gram context. When the LLM predicts another token, the local n -gram context transitions into a new sequence that includes the new token at the end. These transitions from one local context to another are represented by edges on a De Bruijn graph (see Fig [1]).

A De Bruijn graph $B(V, n)$ consists of a vertex set where each unique n -gram (a sequence of n tokens) from a vocabulary of size V corresponds to a distinct node. Directed edges encode the transitions from one n -gram to the next upon appending a new token. For instance, transitioning over bigrams might look like: "Jane has" $\rightarrow$ "has two" $\rightarrow$ "two apples". If the model only requires the previous n tokens in its reasoning to uniquely determine the set of next plausible tokens, the prohibitively large prefix tree of reasoning traces collapses into a set of unique paths on $B(V, n)$.

Crucially, if we prune all the edges in the De Bruijn graph $B(V, n)$ that do not belong to a valid reasoning path, the resulting subgraph must be a Directed Acyclic Graph (DAG). Including only forward-pointing edges in this way ensures that all reasoning paths are finite.

Remark 1 For the purposes of enumerating reasoning pathways in this paper, we ignore "recovery branches" that would point backwards in the DAG and allow a LLM to correct incorrect reasoning. Understanding when and where to place such branches for robust error-correction is a compelling avenue for future work.

We would like to understand how much data it would require for a LLM to learn all or most of the reasoning paths in a large DAG.

The De Bruijn graph $B(V, n)$ contains V^n nodes. We designate N vertices as answer nodes with no outgoing edges (the possible answer choices) and S vertices as start nodes with no incoming edges (the starting sequences). Given a DAG $D_\pi \subset B(V, n)$ that is constructed to maximize the number of distinct start-to-answer paths, we would like to compute (i) the minimum number of samples of reasoning paths required to cover every edge, and (ii) the minimum length of reasoning samples required to cover every edge. The first calculation is a lower bound on the number of training samples a LLM requires to learn every reasoning path while the second calculation bounds how well a LLM can generalize to long-horizon tasks.

Consider the following procedure F to produce a DAG from a De Bruijn graph $B(V, n)$.

1. Order the nodes according to a permutation π .
2. Designate the last N nodes as answer nodes, and the first S nodes as start nodes. Delete outgoing edges from answer nodes and incoming edges from start nodes.
3. Delete any edge pointing from node u to v if v is not later than u in the ordering. An edge from node u to v remains if and only if $\pi(u) < \pi(v)$.
4. Delete any node and edge that does not lie on a start-to-answer path.

We will represent the action of this procedure on a De Bruijn graph as the function $F[\cdot; \pi, N, S]$ where the first argument is the De Bruijn graph, π is a node ordering on that graph, N is the number of answer nodes and S is the number of start nodes. For example, when this procedure is applied to the graph $B(V, n)$ we write $F[B(V, n); \pi, N, S]$. The sum of the start and answer nodes is constrained by the total number of nodes in the graph. $S + N \leq V^n$.

In the following theorems and their proofs, $B(V, n)$ is a De Bruijn graph of size $V > 4$, $n \geq 2$ and the graph $D_\pi := F[B(V, n); \pi, N, S]$.

Lemma 1 D_π is a DAG for any ordering of nodes π .

Proof:

We will prove, by contradiction, that the generated graph has no cycles. Assume that the graph $F[B(V, n); \pi, N, S]$ contains a cycle. Then, it contains a sequence of nodes $(u_1, u_2, \dots, u_k)$ for some integer $k \geq 1$ such that there is a directed edge from node u_i to u_{i+1} for any $1 \leq i < k$, and there is a directed edge from u_k to u_1 . This means that

$\pi(u_1) < \dots < \pi(u_k) < \pi(u_1) \implies \pi(u_1) < \pi(u_1)$ which is clearly an impossibility. By contradiction, the procedure generates a DAG. $\square$

Lemma 2 There exists a number of start nodes $S > 0$ and a permutation π such that D_π contains the maximal count of distinct paths from a start node to the N answer nodes of any DAG that can be constructed from the edges in $B(V, n)$.

Proof:

Let D^* be a DAG that only contains edges from the De Bruijn graph $B(V, n)$, maximizes the number of distinct paths from its S^* start nodes to the N answer nodes and does not contain any vacuous edges or nodes that are not on a start-to-answer path. There exists a sort π^* of the nodes in $B(V, n)$ such the ordering in π^* defines a topological sort over the nodes in D^* , all S^* sources (start nodes) in D^* come first in the ordering and all N sinks (answer nodes) last. Let $D = F[B(V, n); \pi^*, N, S^*]$ be the DAG generated by our procedure using the optimal number of answers S^* and ordering of nodes π^* . This procedure deletes any edge from node u to v such that $\pi^*(u) \geq \pi^*(v)$. Since π^* defines a topological sort of the nodes in D^* , no edge in D^* will be deleted by this procedure. Furthermore, both D and D^* have the same set of start and answer nodes, and D^* does not contain any edges that are not in the de Bruijn graph $B(V, n)$. This means that the generated graph D contains all the edges in D^* , so $D^* \subseteq D$. Therefore the number of distinct paths from start to answer nodes in D is greater than or equal to the number of paths of that kind in D^* . Since D^* is a maximizer of the number of paths, D is as well. $\square$

Lemmas 1 and 2 establish that our procedure F does indeed generate the kinds of reasoning structures we are interested in, i.e., those with a large number of finite-length reasoning traces.

Theorem 1 Let the number of start and answer nodes $S, N < \binom{n+m-1}{m-1}$ for an integer $1 < m < V/2$. There exists a node ordering π such that

1. the total number of start-to-answer paths in D_π is

$$P_{\text{tot}} \geq \sum_{t=0}^{V-2m} \left[\binom{V-2m}{t} \right]^n$$

2. The minimum number of start-to-answer paths required to cover every edge in D_π is

$$P_{\text{min}} \leq \frac{V^n(V+1)}{2}.$$

Proof:

Define the alphabet of size V , $A = \{0, \dots, V-1\}$. Each node u in D_π can be written as $u = a_1 a_2 \dots a_n$ for all $a_i \in A$. We will also use shorthand such as 0^k or $(V-1)^k$ to refer to length k strings of the repeated character 0 or $(V-1)$ respectively. While this shorthand could be mistaken as exponents, its meaning will be stated or clear from the context. Define the digit-sum of this node u as

$$\sigma(u) = \sum_{i=1}^n a_i.$$

And let π be an ordering by this digit sum such that for any two nodes u_1, u_2 in D_π ,

$$\pi(u_1) < \pi(u_2) \implies \sigma(u_1) \leq \sigma(u_2).$$

(1) We will first show the bound on the total number of start-to-answer paths P_{tot} .

Consider the original De Bruijn graph $B(V, n)$ from which the DAG D_π was derived. There is an edge $u \rightarrow v$ in $B(V, n)$ if and only if we can write $u = a_1 a_2 \dots a_n$ and $v = a_2 a_3 \dots b$ for all $a_i, b \in A$. This edge remains in D_π if $\sigma(v) > \sigma(u)$ so $b > a_1$. Taking this one step further, consider the directed path with l nodes, $u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_l$ in $B(V, n)$. This path is generated from the string $a_1 a_2 \dots a_{n+l-1}$ where each $u_j = a_j a_{j+1} \dots a_{n+j-1}$. The path is guaranteed to exist in D_π if (i) u_1 is a start and u_l is a answer, (ii) no intermediate node u_i for $1 < i < l$ is a start or a answer, (iii) and every edge in the path points to a node with a greater digit-sum than the last. The third condition is equivalently rewritten as

$$a_{n+j} > a_j$$

for any $1 \leq j \leq l-1$. This observation will be useful later in the proof.

Recall the start nodes are the first S nodes in π and the answer nodes are the last N nodes in π . Because the number of start and answer nodes are bounded by $\binom{n+m-1}{m-1}$, each start node has digit-sum $< m$ and each answer node has digit-sum $> n(V-1) - m$. This is because there are exactly $\binom{n+m-1}{m-1}$ possible ways to create start or answer nodes with their respective allowed digit-sums (by the stars and bars method). In addition, the node with the smallest digit-sum 0^n is guaranteed to be a start and the node $(V-1)^n$ with the largest digit-sum is guaranteed to be a answer node. Therefore, the number of distinct paths from 0^n to $(V-1)^n$ in D_π is a lower bound on the total number of start-to-answer paths in D_π .

We would like to estimate this value.

A generic path between these two nodes $0^n \rightarrow u_1 \rightarrow \dots \rightarrow u_{tn+n-1} \rightarrow (V-1)^n$ with $(t+1)n-1$ intermediate nodes can be generated from the string $0^n a_1 a_2 \dots a_{tn} (V-1)^n$.

This is a valid path in D_π if

(i) $a_i \notin \{0, 1, \dots, m-1\} \cup \{V-1, V-2, \dots, (V-1)-m+1\}$ which guarantees that no intermediate node in the path is a start or a answer,

(ii) $a_{j+n} > a_j$ for any $1 \leq j \leq n(t-1)$.

These conditions are repeated from our observation above. Satisfying them requires that $0 \leq t \leq (V-2m)$. The number of such paths is just the number of monotonically increasing sequences of length t , raised to the n and summed over all t

$$\sum_{t=0}^{V-2m} \left[\binom{V-2m}{t} \right]^n.$$

(2) Now we will prove the second statement in the theorem for the same ordering π . First note that the minimum number of paths required to cover every edge in D_π , denoted by $P_{\min}$, is at most the number of edges in D_π . The following argument proves this fact.

Consider the set $\mathcal{P}_{\min}$ of paths in D_π that cover all the edges in D_π and is of size $P_{\min}$. Each path $p \in \mathcal{P}_{\min}$ must contain at least one edge in D_π that is not in any other path in $\mathcal{P}_{\min}$.

Otherwise the set $\mathcal{P}_{\min} \setminus \{p\}$ has size $P_{\min} - 1$ and still covers every edge in D_π . This is in contradiction with the definition that $P_{\min}$ is the minimum number of paths required to cover all the edges in D_π . Since every path $p \in \mathcal{P}_{\min}$ is associated with at least one unique edge, $P_{\min}$ can be at most the number of edges in D_π . We would like to upper bound the number of edges in D_π which will give us an upper bound on $P_{\min}$.

For every pair of nodes $u = a_1 \dots a_n$ and $v = b_1 \dots b_n$ for $a_i, b_i \in A$, an edge exists from $u \rightarrow v$ only if $b_n \geq a_1$ and $a_{i+1} = b_i$ for all $1 \leq i \leq n-1$. There are

$$\frac{V^n(V+1)}{2}$$

ordered pairs of nodes satisfying these properties which is an upper bound on $P_{\min}$. $\square$

Corollary 1 The ratio $P_{\min}/P_{\text{tot}} = 0$ in the limit $V \rightarrow \infty$. In addition, $P_{\min}/P_{\text{tot}} \rightarrow 0$ in the limit $n \rightarrow \infty$ provided V is sufficiently large that $\binom{V-2m}{\lfloor (V-2m)/2 \rfloor} > V$.

Proof:

By taking the largest term in the bound on P_{tot} in the first part of theorem 1, we know that

$$P_{\text{tot}} \geq \left[\binom{V-2m}{\lfloor (V-2m)/2 \rfloor} \right]^n.$$

In the asymptotic limit $V \rightarrow \infty$ we apply Stirling's approximation and get

$$P_{\text{tot}} \geq \left(\frac{2}{\pi} \right)^{n/2} \frac{2^{n(V-2m)}}{(V-2m)^{n/2}}.$$

In this limit, the ratio $P_{\min}/P_{\text{tot}}$ decays to 0 exponentially quickly as V increases.

Next consider the same ratio

$$\frac{P_{\min}}{P_{\text{tot}}} \leq \left[\frac{V+1}{2} \left(\frac{V}{\binom{V-2m}{\lfloor (V-2m)/2 \rfloor}} \right)^n \right]$$

in the limit that $n \rightarrow \infty$. For sufficiently large V , the fraction $V/\binom{V-2m}{\lfloor (V-2m)/2 \rfloor} < 1$. Thus, taking the limit $n \rightarrow \infty$ of $P_{\min}/P_{\text{tot}}$ yields 0. $\square$

Corollary 1 shows that when reasoning traces are organized on a De Bruijn graph structure, it is possible to learn a large number of reasoning paths despite seeing very little data.

Theorem 2 Let the number of start and answer nodes $S, N < \binom{n+m-1}{m-1}$ for an integer $1 < m < V/2$. There exists a node ordering π such that,

1. The longest start-to-answer path in D_π has length $L_{\max} \geq n(V - 2m + 1)$.
2. Every edge in D_π lies on a start-to-answer path of length $L_{\text{cover}} \leq 2n + 1$.

Proof:

Let π be the digit-sum ordering as described in the proof of Theorem 1. If two nodes $u = a_1 a_2 \dots a_n$ and $v = b_1 b_2 \dots b_n$ have the same digit-sum, we will break ties by lexicographically comparing their respective reversed strings $\bar{u} = a_n a_{n-1} \dots a_1$ and $\bar{v} = b_n b_{n-1} \dots b_1$. Specifically, $\bar{v} < \bar{u}$ in the lexicographic comparison if for $i^* = \max\{i : a_i \neq b_i\}$, $b_{i^*} < a_{i^*}$. If $\bar{v} < \bar{u}$ lexicographically and both nodes have the same digit-sum, then $\pi(v) > \pi(u)$.

(1) We will start by proving the first bound about the longest start-to-answer path in the statement of the theorem. We can construct a path $u_1 \rightarrow \dots \rightarrow u_L$ based on the string

$$w = w_1 \dots w_{n(V-2m+2)} = 0^n m^n (m+1)^n \dots (V-1-m)^n (V-1)^n$$

where each node $u_j = w_j w_{j+1} \dots w_{j+n-1}$ is the associated slice of the string. The digit-sum is strictly increasing along this path, $\sigma(u_{j+1}) - \sigma(u_j) > 0$, the first node $u_1 = 0^n$ is guaranteed to be a start and the last node $u_L = (V-1)^n$ is an answer. In addition, recall from the proof of Theorem 1 that every start node s has digit-sum $\sigma(s) < m$ and every answer node s' has digit-sum $\sigma(s') > n(V-1) - m$, so no intermediate node u_j for $1 < j < L$ is a start or an answer node. Therefore, the path $u_1 \rightarrow \dots \rightarrow u_L$ is contained in D_π where $L = n(V - 2m + 1) + 1$. This path has length $n(V - 2m + 1)$, proving the first bound in the theorem.

(2) Next we will prove the second bound in the theorem about the length of paths required to cover every edge in D_π . Let $u \rightarrow v$ be an arbitrary edge in D_π . We will show that this edge is contained in a path of length less than or equal to $2n + 1$.

First, a definition. We call an edge $u \rightarrow v$ of $B(V, n)$ **admissible** if it survives steps 2 and 3 of the procedure F , i.e., if u is not an answer node, v is not a start node, and $\pi(u) < \pi(v)$.

Every edge of D_π is admissible, but the converse may not be true due to step 4 in F . In the

following argument, we verify admissibility of edges and defer applying step 4 to the final paragraph.

We start by proving that there exists a path from a start node to $u = a_1a_2 \dots a_n$ that has length less than or equal to n . Consider a sequence of $n + 1$ nodes $(u_0, u_1, \dots, u_n)$ such that $u_0 = 0^n$, $u_n = u$ and each intermediate $u_j = 0^{n-j}a_1a_2 \dots a_j$. Let i_0 be the index of the latest start node in the sequence. We will prove that the path $u_{i_0} \rightarrow u_{i_0+1} \rightarrow \dots \rightarrow u_n$ is a valid path in D_π of length $n - i_0$. If u is a start node, then the path is of length 0 and is trivially in D_π . In addition, no node in the path is a answer node since u has an outgoing edge to v and the intermediate nodes have a digit-sum of at most $(n - 1)(V - 1)$ which is smaller than the minimum answer node digit-sum of $n(V - 1) - m + 1$. Consider a transition $u_{j-1} \rightarrow u_j$ in the path for $j > i_0$. There are two cases.

(i) If $a_j > 0$, then $\sigma(u_j) > \sigma(u_{j-1})$ so $\pi(u_j) > \pi(u_{j-1})$ and the edge $u_{j-1} \rightarrow u_j$ is admissible.

(ii) If $a_j = 0$, we can compare u_j with u_{j-1} using the reverse lexicographic sort. Let $i^* = \max\{i : i < j \text{ and } a_i > 0\}$ which is well-defined for any u_j in the path

$u_{i_0} \rightarrow u_{i_0+1} \rightarrow \dots \rightarrow u_n$ since $u_j \neq 0^n$ for any $j > i_0$. The character a_{i^*} is at position $n - j + i^* + 1$ in node u_{j-1} and it is at the earlier position $n - j + i^*$ in node u_j with every subsequent character being 0, which is smaller than a_{i^*} . This means $\bar{u}_j < \bar{u}_{j-1}$ in the reversed lexicographic sort which we are using for tie-breaking, so $\pi(u_j) > \pi(u_{j-1})$ and, once again, the edge $u_{j-1} \rightarrow u_j$ exists in D_π .

Therefore, every edge of $u_{i_0} \rightarrow u_{i_0+1} \rightarrow \dots \rightarrow u$ is admissible. This means there is a walk of admissible edges from a start node to an arbitrary node u , of length $\leq n$.

Next, we prove that there exists a path from an arbitrary node $v = b_1b_2 \dots b_n$ in D_π to a answer node that has length less than or equal to n . There are two cases.

(i) Each $b_j < (V - 1)$. Consider a sequence of $n + 1$ nodes $(v_0, v_1, \dots, v_n)$ such that $v_n = (V - 1)^n$ is the largest possible node, $v_0 = v$ and each intermediate $v_j = b_{j+1}b_{j+2} \dots b_n(V - 1)^j$. Let k_0 be the index of the earliest answer node in the sequence. We will prove that the path $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{k_0}$ is a valid path in D_π . If v is a answer node, then the path has length 0 and is trivially in D_π . In addition, no node in the path is a start node since v has an incoming edge from u and the intermediate nodes have a digit-sum of at least $V - 1$ which is larger than the maximum start node digit-sum of $m - 1$. Consider a transition $v_{j-1} \rightarrow v_j$ in the path. Since $b_j < V - 1$, $\sigma(v_j) > \sigma(v_{j-1})$ so the edge $v_{j-1} \rightarrow v_j$ is admissible. Therefore, the path $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{k_0}$ is a walk of admissible edges of length $\leq n$.

(ii) There exists a j such that $b_j = V - 1$. Let $i_1 = \min\{i : b_i = (V - 1)\}$. Consider a path $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ for $v_0 = v$ and to a answer node v_k . This path can be generated by a string $b_1b_2 \dots b_{n+k}$ where each $v_j = b_{j+1} \dots b_{j+n}$. We will prove by contradiction that $k \leq i_1$, otherwise the path cannot exist in D_π . Consider $k > i_1$ and assume the path exists in D_π . There is a transition $v_{i_1-1} \rightarrow v_{i_1}$ in the path. This transition can be written in characters as $(V - 1)b_{i_1+1} \dots b_{n+i_1-1} \rightarrow b_{i_1+1} \dots b_{n+i_1}$. Since the transition is represented by an edge in D_π , it must be true that

$\sigma(v_{i_1-1}) \leq \sigma(v_{i_1}) \implies (V-1) \leq b_{n+i_1} \implies b_{n+i_1} = (V-1) \implies \sigma(v_{i_1-1}) = \sigma(v_{i_1})$, resulting in a tie for the digit-sum. Comparing the reverse lexicographic sort yields $\bar{v}_{i_1} > \bar{v}_{i_1-1} \implies \pi(v_{i_1}) < \pi(v_{i_1-1})$ provided $v_{i_1} \neq (V-1)^n$ is not a answer (true by assumption). This is a result of a direct comparison of $v_{i_1-1} = (V-1)c_1 \dots c_{n-1}$ to $v_{i_1} = c_1 \dots c_{n-1}(V-1)$ where at least one of the $c_j < V-1$. This means there cannot be an edge from $v_{i_1-1} \rightarrow v_{i_1}$ contradicting our assertion that the path $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ exists in D_π . Therefore it must be true that $k \leq i_1 \leq n$. It follows that, if there exists a path in D_π from v to a answer node (true by assumption that the edge $u \rightarrow v$ exists in D_π), its length is upper-bounded by n .

If the edge $u \rightarrow v$ exists in D_π , then by the two arguments above there is a walk of admissible edges from a start node to u of length $\leq n$, and a walk of admissible edges from v to an answer node of length $\leq n$. Concatenating these with the edge $u \rightarrow v$ gives a walk W from a start node to an answer node in which every edge is admissible, so π is strictly increasing along W . Hence no node of W repeats and W is a path. Since W runs from a start node to an answer node, no edge of W is deleted by the pruning step 4 in the procedure F , so $W \subseteq D_\pi$. Its length is at most $2n + 1$ and it contains $u \rightarrow v$.

Corollary 2 In the limit $n \rightarrow \infty$, the ratio $L_{\text{cover}}/L_{\text{max}} \leq 2/(V - 2m + 1)$.

Proof:

A straightforward evaluation of the limit $\lim_{n \rightarrow \infty} \frac{2n+1}{n(V-2m+1)}$.

Remark 2 For a constant state space V^n , maximizing the vocabulary size V minimizes the ratio $P_{\text{min}}/P_{\text{tot}}$, i.e., it maximizes the fraction of the reasoning prefix tree that can be reconstructed per sample. This is because P_{tot} grows combinatorially in V and P_{min} grows at most polynomially in V . Large V also minimizes the ratio $L_{\text{cover}}/L_{\text{max}}$. This seems to suggest large vocabularies are optimal for sample efficiency, contradicting the empirical preference for intermediate vocabulary sizes calibrated to model size and data quantity (<https://arxiv.org/abs/2407.13623>). This discrepancy is resolved by considering the complexity of each next-token prediction. The out-degree of any node in D_π increases with V , and larger V increases prediction error by forcing the LLM to distinguish between a large number of plausible next options at every reasoning step (see Appendix A and [Nadgir et al.] for a more in-depth analysis of this kind of error). The optimal vocabulary size is therefore governed by a trade-off between maximizing the sample efficiency of reconstructing the reasoning prefix tree (favoring a large V) and minimizing single-step reasoning errors (favoring a small V). With more data, the LLM has smaller single-step reasoning errors and it should prefer larger vocabulary sizes V . Similarly, human experts develop large and niche vocabularies in their respective fields [<https://pubmed.ncbi.nlm.nih.gov/28787239/>; https://iiif.library.cmu.edu/file/Simon_box00005_fld00354_bdl0001_doc0001/Simon_box00005_fld00354_bdj].

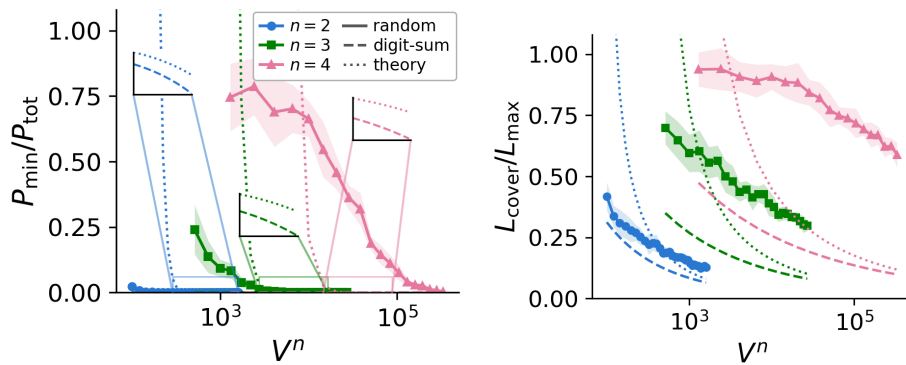


Fig 2: A perfect memorizer can learn long reasoning paths with few and length-limited samples. Results are on DAGs built by applying the procedure F to De Bruijn graphs, with DAG size varied through the context length n and vocabulary size V . **(Left)** The minimum number of start-to-answer paths needed to cover every edge, $P_{\min}$, shrinks relative to the total number of paths P_{tot} as the DAG grows. A small fraction of the set of all paths is sufficient to see every transition. **(Right)** The minimum length of sampled paths needed to cover every edge, L_{cover} , likewise shrinks relative to the longest start-to-answer path L_{max} . Full coverage is achievable with only short samples. **Details:** All DAGs were created with $S = 5$ start nodes, $N = 10$ answer nodes at the topological order π that is either uniformly random (curves averaged over 10 trials; shading is the standard error of the mean) or the digit-sum ordering with lexicographic tie-breaking from Theorems [1] and [2]. The theory curves represent the bounds from Theorems [1] and [2] which hold here for the digit-sum ordering.

Fig [2] visualizes the bounds in Theorems [1] and [2] in DAGs created from applying the procedure F to actual De Bruijn graphs of varying sizes. We see in Fig [2] (Left) that, as the vocabulary size V increases, a perfect memorizer needs few samples to learn every path in the DAG relative to the number of total reasoning paths. This means that, when reasoning paths can be organized on a De Bruijn structure, they can be learned with very few samples. Similarly, Fig [2] (Right) shows that a perfect memorizer would only need to see samples of relatively short paths to learn the entire DAG. Due to the De Bruijn structure in the DAG, it is possible to generalize from short, simple examples to tasks which require much longer reasoning. While these results hold for a hypothetical class of perfect learners, we would like to understand if Transformer models trained on textual sequences with De Bruijn structure see similar gains.

Indeed, Fig [3] shows high sample efficiency and length generalization when Transformer models are trained to reproduce paths with a De Bruijn structure. We trained Transformer models ($\sim 0.8\text{M}$ parameters) on start-to-answer paths sampled uniformly at random until it learned the set of all start-to-answer paths "sufficiently well". Specifically, we evaluated the models on 8000 samples drawn uniformly at random over the set of start-to-answer paths in the DAG according to the following two metrics:

1. **Path coverage:** The proportion of paths with probability, as computed by the trained model, greater than 0.1 of the true path probability.
 2. **P(invalid tokens):** The average, over all next-token predictions in all the sampled paths, of the probability of producing a token that is not one of the allowed branches.
- A model was considered trained if it learned most of the paths in the graph and was unlikely to generate an incorrect path, i.e., if it satisfied path coverage > 0.95 and $P(\text{invalid tokens}) < 0.05$.

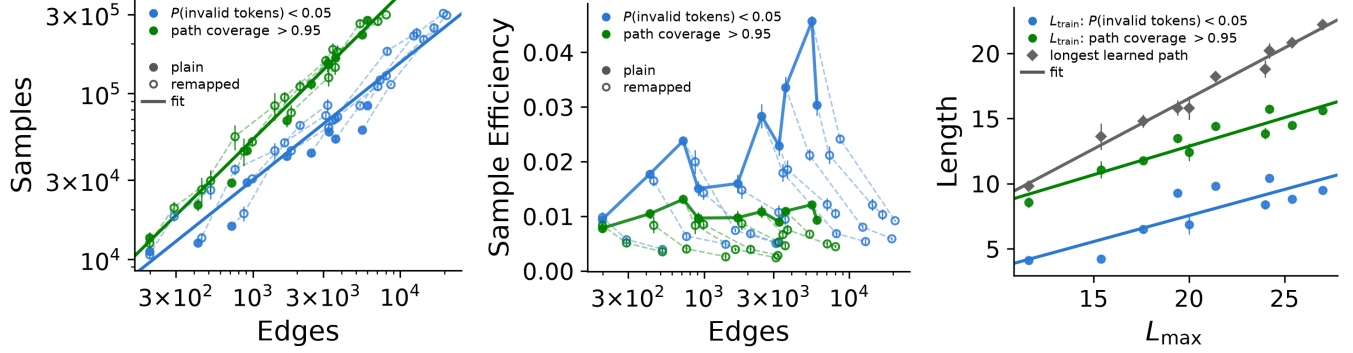


Fig 3: Transformer models can learn most paths on a De Bruijn DAG from few and length-limited samples. Each point averages over DAGs built by applying procedure F to De Bruijn graphs with random orderings π (and, for remapped points, random token permutations). Error bars are standard error of the mean. Each training sample is one start-to-answer path. **(Left)** The samples needed to learn the paths in both original and remapped DAGs scale with the number of distinct edges the model must learn ($\sim x^{0.71}$ for $P(\text{invalid}) < 0.05$; $\sim x^{0.88}$ for path coverage > 0.95). Since the remapped DAGs share their underlying topology, vocabulary, and context length with the originals, edge count is the primary driver of sample cost. **(Middle)** Sample efficiency, i.e., the minimum cover size $P_{\min}$ divided by the samples needed to learn the DAG, is ~ 0.01 for non-remapped points. Remapped points have lower sample efficiency since their paths do not directly expose the underlying topology of the De Bruijn DAG. **(Right)** Models trained only on paths shorter than L_{train} meet both learning criteria at non-trivially small L_{train} and cover longer paths. the gap between L_{train} to meet both learning criteria and the longest path the model covers at that training length grows with DAG size (correlated with $L_{\max}$). More details about the experiment are in Appendix [B].

Fig [3] (Left) shows that the number of samples required to learn the start-to-answer paths in a DAG with De Bruijn structure scales as a power law with the number of edges. This is a significant gain in sample efficiency compared to pure memorization of every possible reasoning path whose number grows combinatorially in the vocabulary size V and exponentially in the context size n (see Theorem [1]). Fig [3] (Middle) corroborates this point and shows that the number of random samples required to learn the set of start-to-answer paths is only 100x the minimum number of paths required to see every edge (sample efficiency in the plot mostly remains near or above 0.01; a perfect memorizer can achieve efficiency 1). In addition to learning with few samples, we would ideally like our models to generalize to longer and more

difficult tasks than what they have been trained on. Fig [3] (Right) shows that models can learn the set of start-to-answer paths with training data of bounded length. They assign non-trivial probability to paths that are significantly longer than anything they have been trained on.

Fig [3] (Left, Middle) also contain "remapped" points which were included to isolate exactly what aspect of the larger graphs (more edges) caused learning them to require more samples. It could very well be the case that larger graphs have a non-trivially different underlying topology, or perhaps the samples required for learning actually scales with another correlated variable instead (e.g., vocabulary size). To understand whether the number of samples required for learning truly scaled with the number of edges in the graph, we converted each DAG into three other remapped DAGs with the same underlying topology, vocabulary size, etc. as the original graph. We varied the number of distinct edge transitions from past context to next-token by permuting the token labels at regular intervals of length 4, 8 and 16. For example, in a binary sequence if the token labels $1 \rightarrow 0$ and $0 \rightarrow 1$ after a length 2 interval, the sequence $1011 \rightarrow 1000$ and $0110 \rightarrow 0101$. A model learning the original set of sequences would only have to learn the transitions in the original De Bruijn DAG. A model learning the remapped sequences would have to learn, not only the original transitions, but also a different set of transitions to use at each length interval. The later transitions have the same underlying topology as the original De Bruijn DAG but with different node labels. Fig [3] (Left) shows that the number of samples required to learn these remapped sequences scales with the number of distinct transitions (edges) the model has to learn, regardless of if the remapped sequences share a simpler underlying topology with the original De Bruijn DAG. This is why Fig [3] (Middle) shows that sample efficiency decreases for the remapped points compared to the original, since the remapped sequences require more samples for the model to learn them, but a perfect memorizer would only need to learn the topology of the original De Bruijn DAG. Thus, regardless of the underlying structure that generated the data, it is the number of distinct transitions, or edges, that the model must learn which controls the number of samples required.

This has some important consequences. Consider an arbitrary set of reasoning traces to autoregressively solve a task. They can be organized on a "minimal DAG" such that each node in this DAG is labeled by the necessary tokens from past context to exactly determine the set of possible suffixes, and each edge is labeled by the next-token prediction to transition into a new context. Any two nodes with the same label must also share suffixes and are therefore merged into the same node. The number of edges in this DAG is the minimum number of transitions from context to next-token that a language model would have to learn in order to learn the set of reasoning paths. The results in Fig [3] (Left, Middle) indicate that the number of training samples required to learn the reasoning paths scales with the number of edges in the minimal DAG. This is an issue if the start-to-answer paths on the DAG actually follow the transition rules of a simpler underlying world-model. In that case, it is desirable to learn the set of reasoning traces with much fewer samples than the number of edges in the inefficient DAG representation. Past research has already shown that standard next-token prediction is not

sufficient to efficiently learn the underlying world-model of a navigation task if it is not explicitly represented in the training data

[https://proceedings.neurips.cc/paper_files/paper/2024/file/2f6a6317bada76b26a4f61bb70a7db59-Paper-Conference.pdf]. In Fig [4], we show that encoding navigation directions using a De Bruijn structure leads to significant improvements in test accuracy.

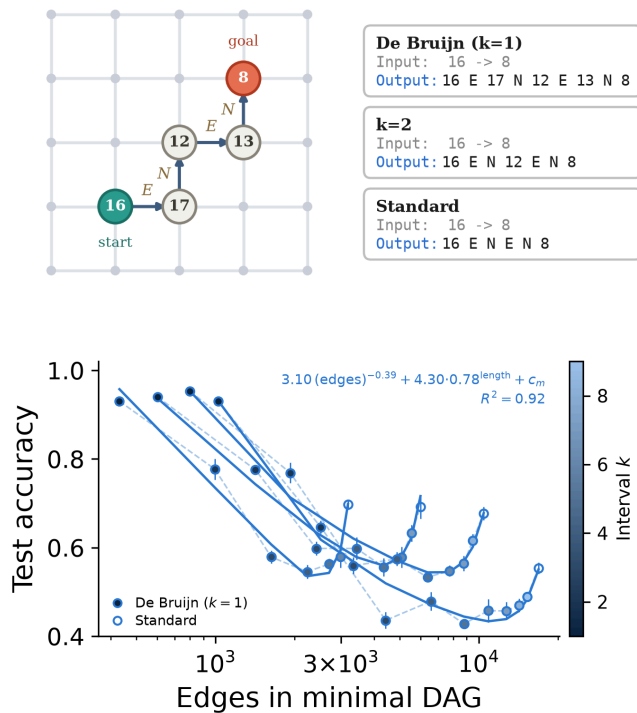


Fig 4:

Split error into

Error of learning world + error of classification

Average trace lengths (answer tokens):

GSM8K 4407 sequences | mean trace length = 153.9 tokens

MATH-500 363 sequences | mean trace length = 274.4 tokens

GPQA 170 sequences | mean trace length = 236.5 tokens

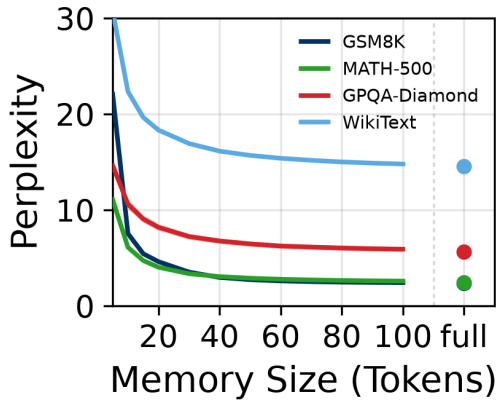
WikiText 3176 sequences | mean trace length = 179.4 tokens

'--knockout-ns', type=str, default="5,10,15,20,30,40,50,60,70,80,90,100"

SEM errorbars are too small to see

filtered out traces of length shorter than 100

wiktext offset: 59 tokens, all wiktext traces have length ≥ 159



Section 2: There is more uncertainty in the early phases of reasoning compared to later

It is interesting to understand the structure of the reasoning prefix tree that is generated by applying the procedure F to the De Bruijn graph $B(V, n)$. We will see that nodes which appear deeper in the tree tend to have a smaller out-degree. Such nodes will also generally appear later in the topological sort of the DAG created by applying F , and Theorem 3 states that such nodes have a smaller out-degree compared to earlier ones.

Theorem 3 In expectation over node orderings π sampled uniformly at random,

$$\mathbb{E}_{\pi}[\deg^+(\pi^{-1}(d))] \leq V \left(1 - \frac{d}{V^n}\right)$$

where $\deg^+(u)$ is the number of outgoing edges for node u and d is the node's position in the sort.

Proof:

We will adopt the notational conventions from the proof of Theorem 1, where A is the alphabet of size V , and $a \in A$ are characters. Each node u in $B(V, n)$ can be written as $a_1 a_2 \dots a_n$ where each $a_i \in A$. The out-neighbors of u in $B(V, n)$ are $v_b = a_2 \dots a_n b$ for any $b \in A$. There are V such neighbors. Let $c(u)$ count the outgoing branches of a node u in $B(V, n)$. We exclude self-loops from this count because the DAG D_{π} cannot contain any. Therefore, $c(u) = V - 1$ for nodes of the form $u = a^n$ since there is one edge from u to itself. There are V such nodes. For all other nodes, $c(u) = V$. Averaged over both kinds of nodes,

$$\frac{1}{V^n} \sum_{u \in B(V, n)} c(u) = \frac{V(V-1) + V(V^n - V)}{V^n} = \frac{V^n - 1}{V^{n-1}}.$$

In addition, for any node $v \neq u$, the probability that $\pi(v) > \pi(u)$ is

$$\Pr[\pi(v) > d | \pi(u) = d] = (V^n - d) / (V^n - 1)$$

by exchangeability of the nodes v . Consider a DAG D'_π which is generated by applying a modified procedure F' that is identical to F without the steps 2 or 4 which delete edges from answer, start and disconnected nodes. In this modified DAG D'_π , the out-degree of the node at position d , $\deg_{\text{modified}}^+(\pi^{-1}(d))$ in expectation over random orderings π is simply the product

$$\mathbb{E}_\pi[\deg_{\text{modified}}^+(\pi^{-1}(d))] = \frac{V^n - 1}{V^{n-1}} \cdot \frac{V^n - d}{V^n - 1} = V \left(1 - \frac{d}{V^n}\right)$$

due to linearity of expectation. Since $D_\pi \subseteq D'_\pi$ (steps 2 and 4 in F delete edges), the quantity above serves as an upper bound on the expected out-degree of the nodes in D_π ,

$\mathbb{E}_\pi[\deg^+(\pi^{-1}(d))]$. $\square$

The initial "exploration" phase of reasoning generally has more uncertainty than later phases. We can think of this initial phase as setting the problem-solving approach for the rest of the reasoning sequence, after which the steps are more straightforward to determine. In actual LLMs, the initial reasoning steps also have much more uncertainty compared to later steps. Once the LLM finds an appropriate strategy to solve the problem, it converges to a solution (see the following figure from my previous paper)

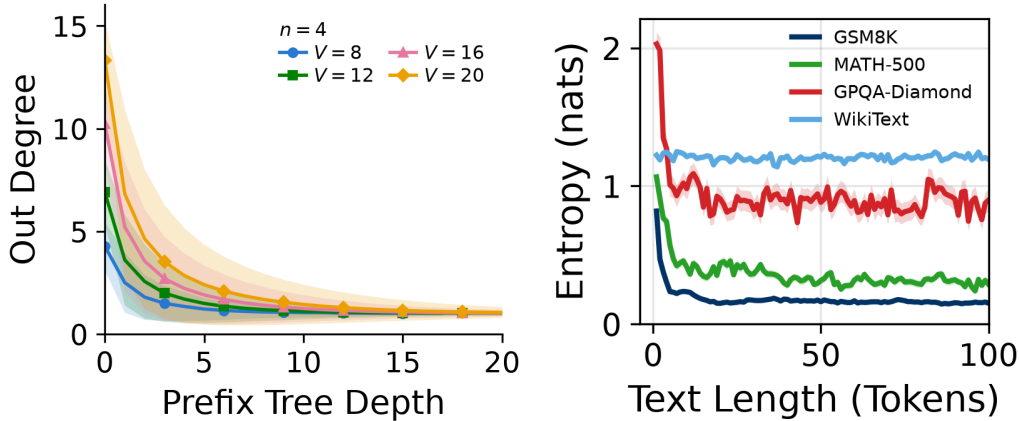


Fig: (Left) Average out-degree of the unrolled prefix tree versus depth over random samples of the topological sort π . answer nodes, with an out-degree of zero, are not counted in the average. (Right) Perplexity when reasoning decays over time, indicating there is greater uncertainty in the initial reasoning steps. wikitext offset: 59 tokens (average gsm8k prompt length)

Confirming our hypothesis, conditional entropy that decreases over time is strongly correlated with correct answers [<https://arxiv.org/pdf/2508.20395>]. Increasing entropy might correspond to the model taking "recovery pathways" back to earlier reasoning stages if it went down an incorrect path. This indicates uncertainty in the final answer.

Section 3: Discussions

Self-growing reasoning trees - we studied finite sets of reasoning paths. When developing something like a theory, you need to add your new results to your existing reasoning and build

on them. E.g., new definitions and theorems can increase your vocabulary and reveal shortcut paths to existing results as well as new paths to deeper results. If you do not learn from your own results, there is a limit to how much you can reason that is set by your limited n -gram context and vocabulary. "Is there a universal, simple and efficient learning algorithm"

You should study recovery paths by including cycles in the de bruijn graphs. Similar to how it is done in robotics and self-driving car, where if the model makes a wrong or slightly sub-optimal decision, it still has a path to recover from its mistake and continue reasoning. Cycles can induce infinite loops, where a model gets stuck, but also allows for robustness to errors. When/where to include cycles is a direction for future research. It is also interesting to study how using external verifiers can trigger these loops or modify the structure of the tree.

Difference between the statistics of effective test-time scaling and of communicating a reasoning trace. Communication should be redundant so actors with different reasoning trees can identify where in the tree they are.

Do humans build neural pathways with De Bruijn structure?

Do these results show that hierarchical reasoning is required for effective generalization?

Generally, we see that early phases of reasoning correspond to more uncertain, higher-level decisions before converging on a strategy. Is this because of an underlying De Bruijn structure in human chains of thought?

The paths in De Bruijn graphs show rich dynamical structure. This indicates that LLMs which learn these paths could have implicitly learned to represent a variety of dynamical time series [<https://openreview.net/forum?id=WwxELkT48R>]. The kinds of time series used to train robots, or in other forecasting tasks, may have the same kind of structure, where most of the next action is predicted using only a limited past context.

Section 5: Conclusion

There has been a lot of back-and-forth discourse about whether LLMs "really reason". Answering this question in any significant way requires a theoretical understanding of what kinds of structures a LLM should have learned to develop the ability to reason effectively. It is also important when designing more efficient test-time scaling in domains such as biology. This paper studies exactly that question and develops the field of theoretical research related to reasoning. The graph-theoretic approach put forth in this paper seems promising so far and may lead to a comprehensive theory. A deep understanding of reasoning should allow us to train LLMs with precise engineering targets about quantities like error robustness, admitting when the answer is unknown, reasoning effort and efficiency, etc.

Appendix A: Predicting non-smooth maps between prompts and answers requires deep reasoning

Consider a task where an LLM is given a prompt and selects one of many possible answers. This is a classification problem, which the model solves by predicting a probability distribution over the next token (this is a simplification, since many answers are multiple tokens long). When LLMs reason using CoT, they are solving a sequence of such classification problems where, at each step, the model must choose the correct next token from a limited set of plausible options.

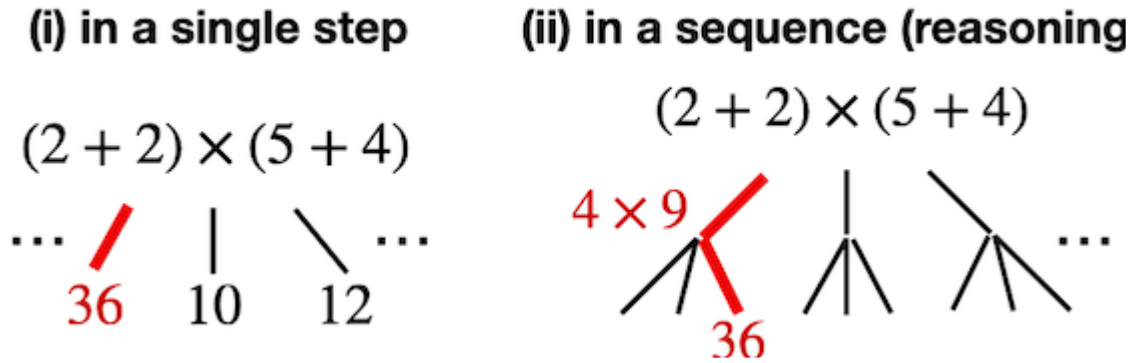


Fig: The set of plausible reasoning sequences **forms a tree**

Say this task has N equally likely plausible outputs, e.g., AIME problems require an integer between 0 and 999, so $N = 1000$. Then, the input domain can be separated into at least N regions that correspond to each output. As N increases, the number of these regions in the input domain does as well, their volume decreases, and they require correspondingly more data to resolve.

Remark The prompts to language models typically come in the form of discrete sentences, but for the purposes of this analysis we think of prompts as embedded in a continuous input space. This modeling choice is justified by a notion of "similarity" between prompts that indicate the presence of an underlying metric structure, such as that in semantic spaces. This kind of structure is required for generalization to be possible at all. In addition, the total number of possible prompts is significantly larger than the amount of data available for training. This means the set of points in the input domain corresponding to all possible prompts is much more dense than the set of data points, to the point where it may as well be continuous.

[Nadgir et al.] proved using Lipschitz arguments that the expected probability of error of a classification task is a power law with respect to the number of distinct regions in the input domain for that task, proportional to N

$$E_{\text{direct}} \propto \left(\frac{N^2}{D} \right)^{1/d}$$

where D is the number of i.i.d. data samples and d is the dimension of the input space.

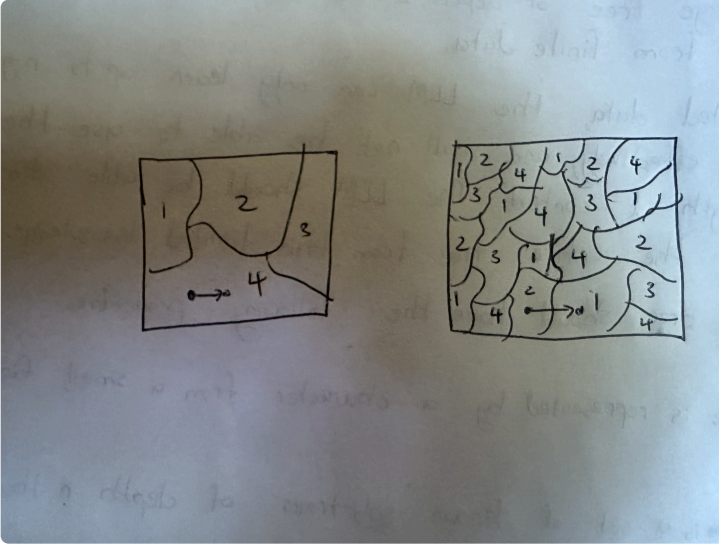


Fig: When there are many regions corresponding to any single answer in the input domain, a small change in the input can cause the output to change.

We consider a closely related setting where the reasoning task is non-smooth such that for every output, there are M distinct regions in the input domain that correspond to that output (see fig). Then, the LLM must distinguish between MN distinct regions in the input domain to determine the correct output class. The corresponding classification error is

$$E_{\text{direct, non-smooth}} \propto \left(\frac{M^2 N^2}{D} \right)^{1/d}.$$

This error can be quite large if M is. CoT helps reduce the error by decomposing this classification problem into a sequence of problems that can each be solved with smaller error. Consider a LLM which generates a CoT of length $\ell - 1$ before predicting the final answer at step ℓ . At any step j in its reasoning sequence it has a choice of, say m_j plausible next tokens. The total number of distinct reasoning paths is denoted by $K = \prod_{j=1}^{\ell} m_j$.

The number of distinct reasoning paths K must be larger than the number of outputs N but smaller than the number of distinct regions in the input space MN . If the number of distinct paths K were larger than MN , we would be separating the input space into more regions than necessary for prediction. Each reasoning path corresponds to, on average, $(MN)/K$ distinct regions in the input space. We would like to find the optimal value for the number of reasoning paths, K . Applying the scaling law we derived above to each step tells us that the error of a step $j \sim (MNm_j/K)^{2/d}$ since there are m_j plausible outputs, each of which corresponds to $(MN)/K$ regions in the input space.

Applying the union bound to compute the error of reasoning over many steps yields

$$E_{\text{reason, non-smooth}} \leq cD^{-1/d} \left(\frac{MN}{K} \right)^{2/d} \sum_{j=1}^{\ell} m_j^{2/d}$$

for a constant of proportionality c .

We can optimize this bound with respect to the number of reasoning paths K , the number of plausible next tokens at each step m_j and the reasoning depth ℓ , subject to the constraint that the total number of paths $\prod_{j=1}^{\ell} m_j = K$. The optimum occurs when each $m_j = \exp(d/2)$ for any step j . In addition, the number of reasoning paths K should be as large as possible so $K = MN$. The optimal reasoning depth $\sim \log K = \log M + \log N$. Therefore, if the true task is not smooth (M is large), the number of distinct reasoning paths is also large. To learn these long and diverse reasoning paths from limited data, the model must rely on local structure in the reasoning.

Appendix B: Experimental details for the synthetic experiment in Fig 3

We trained small ($\sim 0.8M$ parameter) decoder-only Transformer models to auto-regressively predict token sequences, each of which corresponds to a start-to-answer path on a DAG with De Bruijn structure. These DAGs were created by applying the procedure F on De Bruijn graphs of varying vocabulary sizes V and context length n . The number of start nodes S and answer nodes N scaled with the size of the De Bruijn graph, and our results were robust to any particular choices for both values. The topological sorts π were chosen uniformly at random for five replicate experiments per De Bruijn graph. For each of these DAGs that were generated using the procedure F , we created three corresponding remapped DAGs where token values were remapped at random once every 4, 8 or 16 edges. The three remapped DAGs were used for Fig [3] (Left, Middle).

Our experiments used the following conditions:

V	n	S	N
8	2	5	8
10	2	6	10
12	2	8	12
8	3	10	20
10	3	12	24
11	3	14	28
12	3	15	30
8	4	12	24
9	4	14	28
10	4	16	32

For each DAG, we trained a decoder-only pre-norm Transformer model to predict its start-to-answer paths from scratch. The model vocabulary size was equal to the vocabulary size of the DAG with an additional special end of sequence token. It used a linear embedding layer with embedding dimension 128 followed by 4 subsequent Transformer Blocks and then a root-mean-squared norm (RMSNorm) before the output linear layer that predicts logits. The input embedding, output linear layer and RMSNorm weights were all trainable. Each Transformer block consisted of, first, a RMSNorm followed by a multi-head attention with Rotary Positional Embeddings (RoPE) and a residual addition between the output of the attention module with the original input to the Transformer block. The attention module used 4 heads of dimension 32 each. The RoPE used wavelengths of $10000^{(2k)/d_k}$ where $d_k = 32$ is the dimension of the key and query vectors and k is half the feature index, rounded down to the nearest integer. Following the first residual connection, there was a second RMSNorm and then a Swish-Gated Linear Unit (SWiGLU) with a feed-forward dimension 341. Finally, there was a second residual connection which added the output of the SWiGLU block to the output of the first residual connection after the attention computation.

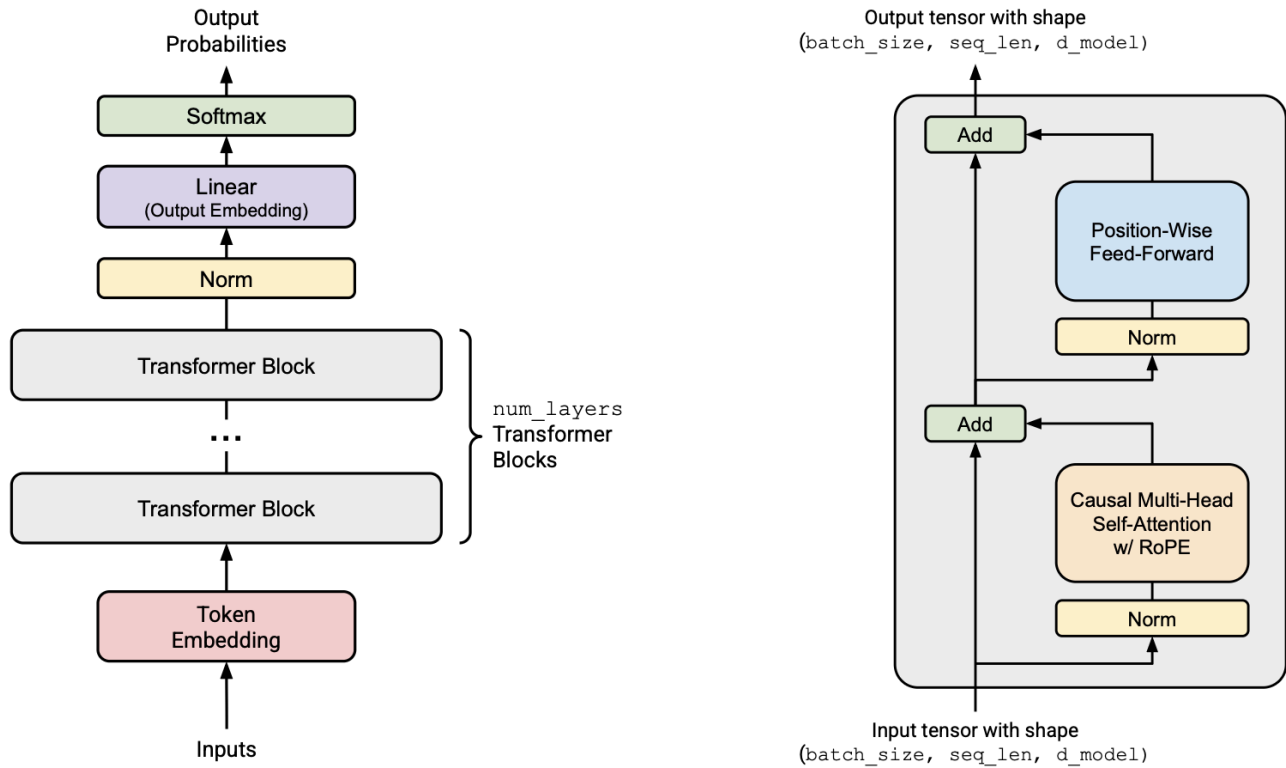


Fig B1 Our model architecture and this depiction of it were taken from the [Stanford CS336 Spring 2026 course materials]. It is a standard pre-norm Transformer architecture.

To evaluate the models, we drew 8000 samples uniformly at random over the set of start-to-answer paths in the DAG and used them to compute the following two metrics:

1. **Path coverage:** The proportion of paths with probability, as computed by the trained model, greater than 0.1 of the true path probability.

2. **P(invalid tokens)**: The average, over all next-token predictions in all the sampled paths, of the probability of producing a token that is not one of the allowed branches.

A model was considered "trained" if it learned most of the paths in the graph and was unlikely to generate an incorrect path, i.e., if it satisfied path coverage > 0.95 and $P(\text{invalid tokens}) < 0.05$.

For training, we used the AdamW optimizer with learning rate 0.001, weight decay 0.01, betas (0.9, 0.98) and batch size 512. To create Fig [3] (Left, Middle), we trained the model for one epoch and evaluated both Path coverage and $P(\text{invalid tokens})$ at regular geometrically-spaced intervals between 316 samples and 316,228 samples with 22 intermediate points (each point was roughly 1.4x the previous). Training was stopped at the intermediate point where both criteria were met: path coverage > 0.95 and $P(\text{invalid tokens}) < 0.05$. Then, we used linear interpolation to estimate the exact number of samples at which these thresholds were crossed. The geometric intervals were chosen to be sufficiently tight that increasing the number of intermediate points did not have any significant impact on the results. To create Fig [3] (Right), the model was trained for 12 epochs on 64,000 training samples that were drawn uniformly at random from the set of start-to-answer paths given that they had length smaller than the training cutoff L_{train} . To encourage length generalization, we augmented the positional indices in each training sequence to include a gap between two adjacent tokens that were chosen uniformly at random over the sequence. The size of the gap was also chosen uniformly at random and was between 1 (no gap) and $L_{\text{max}} - L_{\text{path}}$ where L_{max} is the maximum length of a path in the DAG and L_{path} is the length of the training path. We evaluated path coverage and $P(\text{invalid tokens})$ on every value of L_{train} until both criteria were met and additionally plotted the longest covered path for that training length.

We ran training and evaluation on a cluster of six 4090 RTX GPUs.